



Testing y Buenas Prácticas en ReactJS

Construyendo aplicaciones robustas y mantenibles

Carlos Rojas Sánchez

Ingeniería en Tecnologías Computacionales

Universidad del Mar

Introducción al Testing

¿Qué es el Testing de Software?

Es el proceso de verificar que nuestro código se comporta como esperamos.

- **Verificación:** ¿Estamos construyendo el producto correctamente?
- **Validación:** ¿Estamos construyendo el producto correcto?
- **Prevención:** Encontrar errores antes que el usuario.

Pruebas Unitarias

Se centran en funciones puras o lógica que no depende de la UI.

```
// sum.js  
export const sum = (a, b) => a + b;
```

```
// sum.test.js  
import { sum } from './sum';  
  
test('suma 1 + 2 para igualar 3', () => {  
  expect(sum(1, 2)).toBe(3);  
});
```

En React, estas son las más importantes. Verifican que un árbol de componentes funcione correctamente.

- ¿Se renderiza el hijo dentro del padre?
- ¿Las props pasan correctamente?
- ¿El estado compartido funciona?

Jest es el "Test Runner" más popular en el ecosistema JavaScript/React.

- **Matchers:** `toBe()`, `toEqual()`, `toContain()`.
- **Mocks:** Capacidad de simular módulos o funciones.
- **Snapshots:** Captura la estructura de la UI para evitar cambios accidentales.
- **Watch Mode:** Ejecuta tests automáticamente al guardar.

Pruebas unitarias con Jest

¿Qué es Jest?

Jest es un framework de testing de JavaScript mantenido por Meta, diseñado con un enfoque en la simplicidad y el rendimiento.

- **Zero Config:** Funciona en la mayoría de proyectos React sin configuración inicial.
- **Caja de herramientas completa:** Incluye test runner, librería de aserciones (expect) y utilidades para mocking.
- **Velocidad:** Ejecuta los tests en paralelo utilizando workers independientes.

Estructura básica de un archivo de pruebas

Los bloques `describe` e `it/test` organizan el reporte de ejecución.

```
describe('Grupo de pruebas: Calculadora', () => {  
  
  test('debería sumar dos números', () => {  
    // Código del test  
  });  
  
  it('debería restar dos números', () => {  
    // 'it' es un alias de 'test' (lee como prosa)  
  });  
});
```

Anatomía de una Aserción

Las aserciones validan el resultado usando la función `expect()` combinada con un **Matcher**.

```
expect(resultado).toBe(esperado);
```

```
^^^^^  ^^^^^^^  ^^^  ^^^^^^^
```

Sintaxis	Valor	Matcher	Valor
de Jest	Real		Esperado

Si el valor real no coincide con el esperado, Jest detiene la prueba y genera un reporte detallado del fallo.

Matchers: Valores Primitivos

Para comparar valores exactos o primitivos (booleans, strings, números):

```
test('matchers básicos', () => {  
  expect(1 + 1).toBe(2); // Igualdad estricta (===)  
  
  expect(true).toBeLessThan(2);  
  expect('React').toContain('act');  
  
  expect(null).toBeNull();  
  expect(undefined).toBeUndefined();  
});
```

Matchers: Objetos y Estructuras complejas

`toBe` falla con objetos porque compara la referencia en memoria.
Usamos `toEqual`.

```
test('comparar objetos', () => {  
  const data = { uno: 1 };  
  data['dos'] = 2;  
  
  // expect(data).toBe({ uno: 1, dos: 2 }); // Falla!  
  
  expect(data).toEqual({ uno: 1, dos: 2 }); // Pasa  
});
```

Negar afirmaciones con .not

Cualquier matcher puede invertirse fácilmente encadenando la propiedad `.not`.

```
test('comprobar el caso contrario', () => {  
  const codigo = 'JEST';  
  
  expect(codigo).not.toBe('REACT');  
  expect(10).not.toBeNull();  
  expect([1, 2]).not.toContain(3);  
});  
^^I
```

Ciclos de vida: Setup y Teardown

Jest permite ejecutar código antes o después de los tests para preparar el entorno.

```
beforeAll(() => { /* Corre 1 vez antes de todo */ });  
beforeEach(() => { /* Corre antes de CADA test */ });  
  
afterEach(() => { /* Corre después de CADA test */ });  
afterAll(() => { /* Corre 1 vez al final de todo */ });
```

Útil para limpiar bases de datos simuladas, resets de estados o variables globales.

Ejemplo práctico de Setup

```
describe('Carrito de compras', () => {  
  let carrito = [];  
  
  beforeEach(() => {  
    carrito = ['manzana', 'plátano']; // Reset limpio  
  });  
  
  test('debe añadir un elemento', () => {  
    carrito.push('pera');  
    expect(carrito.length).toBe(3);  
  });  
  
  test('debe mantener tamaño inicial', () => {  
    expect(carrito.length).toBe(2); // No afectada  
  });  
});
```

Testing Asíncrono: Promesas

Para probar funciones que devuelven promesas, podemos retornar la promesa directamente en el test.

```
// fetchUser.js devuelve una Promesa
test('resuelve con el usuario correcto', () => {
  return fetchUser(1).then(data => {
    expect(data.name).toBe('John');
  });
});
```


Testing Asíncrono: Async/Await

La forma moderna y recomendada. Solo añade la palabra clave `async` a la función del test.

```
test('obtiene datos con async/await', async () => {  
  const data = await fetchUser(1);  
  expect(data.id).toBe(1);  
});
```

```
test('maneja errores asíncronos', async () => {  
  await expect(fetchUser(99)).rejects.toThrow();  
});
```

Funciones Mock (Espías)

Los Mocks permiten simular el comportamiento de funciones reales, rastreando sus llamadas.

```
test('rastrear llamadas de un callback', () => {  
  const miMock = jest.fn();  
  
  miMock('hola', 2026);  
  
  expect(miMock).toHaveBeenCalled();  
  expect(miMock).toHaveBeenCalledTimes(1);  
  expect(miMock).toHaveBeenCalledWith('hola', 2026);  
});
```

Mocking: Controlando Valores de Retorno

Podemos inyectar respuestas simuladas a las funciones mock para testear flujos específicos.

```
test('configurar respuestas del mock', () => {  
  const mockCalculador = jest.fn();  
  
  mockCalculador  
    .mockReturnValueOnce(10)  
    .mockReturnValue(0); // Por defecto  
  
  expect(mockCalculador()).toBe(10);  
  expect(mockCalculador()).toBe(0);  
  expect(mockCalculador()).toBe(0);  
});
```

Mocking de Módulos Completos

Útil para interceptar librerías externas como **axios** y evitar peticiones HTTP reales.

```
import axios from 'axios';
import { getUsername } from './userService';

jest.mock('axios'); // Intercepta el módulo

test('debe simular petición http con axios', async () => {
  axios.get.mockResolvedValue({ data: { name: 'Ana' } });

  const name = await getUsername(1);
  expect(name).toBe('Ana');
  expect(axios.get).toHaveBeenCalledWith('/user/1');
});
```

Pruebas de Errores y Excepciones

Para verificar si una función lanza un error, debemos envolverla dentro de otra función reactiva.

```
const romperCodigo = () => {  
  throw new Error('ID no válido');  
};
```

```
test('lanza un error al fallar', () => {  
  // expect(romperCodigo()).toThrow(); // MAL
```

```
expect(() => romperCodigo()).toThrow(); // BIEN  
expect(() => romperCodigo()).toThrow('ID no válido');
```

- **Aislamiento Total:** Usa `jest.clearAllMocks()` en el `afterEach` si reutilizas espías para evitar contaminación de contadores.
- **Prueba Casos Límite (Edge Cases):** No pruebes solo el camino feliz; pasa valores `null`, `undefined` o arrays vacíos.
- **Mantén los Tests Deterministas:** Evita el uso de lógica dinámica basada en la fecha actual (`new Date()`) o números aleatorios sin mockear.

Pruebas de Componentes con Herramientas Especializadas

Las pruebas de componentes verifican el comportamiento de módulos aislados.

Objetivos principales:

- **Shift-Left Testing:** Detectar errores en etapas tempranas.
- **Aislamiento:** Reducir la complejidad de la depuración.
- **Robustez:** Validar límites y entradas inválidas.

Beneficios de la Especialización

El uso de herramientas dedicadas transforma el flujo de trabajo:

Repetibilidad Ejecución instantánea tras cada cambio de código.

Simulación Capacidad de aislar dependencias complejas (APIs, BDs).

Métricas Generación automatizada de reportes de cobertura.

Técnicas de Aislamiento (Test Doubles)

Para probar un componente de forma pura, sustituimos sus dependencias:

- **Stubs:** Proveen respuestas preconfiguradas fijas.
- **Mocks:** Objetos programados para verificar expectativas de llamadas.
- **Spies:** Stubs que registran el historial y parámetros de ejecución.

Ecosistema de Herramientas Especializadas

Cada entorno tecnológico cuenta con soluciones maduras en la industria:

Entorno	Framework de Pruebas	Herramienta de Mocking
Java	JUnit 5	Mockito
JavaScript	Jest / Vitest	Vitest Mocks / Sinon
Python	PyTest	Unittest.mock
C#	xUnit	Moq

Table 1: Herramientas estándar del mercado.

Ejemplo Práctico: Prueba con Jest

A continuación se muestra un componente JavaScript aislado mediante un Mock:

```
// Componente a probar
const calcularTotal = (carrito, servicioDescuento) => {
  const desc = servicioDescuento.obtenerDescuento();
  return carrito.total - (carrito.total * desc);
};

// Prueba unitaria con Jest
test('Aplica 10% de descuento correctamente', () => {
  // 1. Crear el Mock de la dependencia
  const mockServicio = {
    obtenerDescuento: () => 0.10
  };
  const carrito = { total: 100 };

  // 2. Ejecutar y verificar
  const resultado = calcularTotal(carrito, mockServicio);
  expect(resultado).toBe(90);
});
```

Análisis de Cobertura (Code Coverage)

Las herramientas especializadas miden la eficacia de nuestra suite de pruebas:

Métricas Críticas:

- Line Coverage
- Branch Coverage (`if/else`)
- Function Coverage

Buena Práctica: Integrar umbrales mínimos (ej. $> 80\%$) directamente en los pipelines automatizados para bloquear código sin testear.

Integración en el Pipeline de CI/CD

El ciclo automatizado asegura que ningún componente roto llegue a producción:

1. **Fase Local:** Ejecución de tests mediante *git hooks* antes del commit.
2. **Fase Cloud:** Servidores de CI (GitHub Actions, GitLab CI) replican el entorno.
3. **Evaluación:** Si un solo componente falla, el despliegue se detiene de inmediato.

Linting y Estándares de Código

¿Qué es el Linting?

El linting es el proceso de ejecutar una herramienta que analiza el código fuente en busca de errores programáticos, bugs potenciales, inconsistencias de estilo y malas prácticas.

¿Por qué es fundamental?

- **Prevención de errores:** Detecta variables no usadas, código inalcanzable o errores de sintaxis antes de compilar/ejecutar.
- **Consistencia:** Asegura que todo el equipo de desarrollo escriba código con el mismo estilo visual y semántico.
- **Mantenibilidad:** Un código estandarizado es mucho más fácil de leer, revisar y auditar por otros desarrolladores.

Diferencia Crucial: Linters vs. Formateadores

A menudo se confunden, pero operan en capas distintas del análisis de código:

Linters (Análisis Semántico) Buscan problemas de calidad, lógica y reglas de adhesión al lenguaje (ej. evitar el uso de `eval()`, variables globales accidentales).

Formateadores (Análisis Estético) No cambian la lógica; solo modifican el aspecto visual del archivo (ej. saltos de línea, uso de comillas simples vs. dobles, indentación con espacios o tabs).

Herramientas de linting y formateo estándar según la tecnología:

Lenguaje	Linter Principal	Formateador
JavaScript / TS	ESLint	Prettier
Python	Flake8 / Ruff	Black
Java	Checkstyle	Google Java Format
Go	Go Vet	Gofmt

Table 2: Estándares de herramientas en la industria actual.

Ejemplo de Reglas de Linting (Configuración)

Las herramientas se personalizan mediante archivos de configuración (ej. `.eslintrc` o `pyproject.toml`).

Ejemplo conceptual en formato JSON para definir restricciones de estilo:

```
{
  "rules": {
    "no-console": "warn",
    "semi": ["error", "always"],
    "quotes": ["error", "single"],
    "max-lines-per-function": ["warn", 50],
    "no-unused-vars": "error"
  }
}
```

¿Cómo actúa el Linter? (Código Sucio vs. Limpio)

El linter analiza estáticamente el archivo e interrumpe el flujo si encuentra discrepancias.

```
// ALERTA: Código que rompe estándares
function calcular(a,b){
  var x = 10 // Error: usar 'var', falta ';'
  console.log(a) // Advertencia: no dejar 'console.log'
  return a+b;
}
```

```
// CÓDIGO CORREGIDO
function calcular(a, b) {
  const x = 10;
  return a + b;
}
```

Para que los estándares funcionen, el factor humano debe automatizarse:

1. **En el IDE:** Extensiones que formatean el código automáticamente al guardar el archivo.
2. **Pre-commit Hooks (Husky / Pre-commit):** Un script local impide que el desarrollador cree un *commit* si el linter encuentra errores.
3. **CI/CD Gatekeeper:** El servidor de integración continua rechaza cualquier *Pull Request* que no cumpla con el 100% de las reglas estipuladas.